Reviewer Pack — Oracle Collapse of κ: Anti-Relativization Stress Test for th…

Entry a81b4af0bc2b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 17:34:08 UTC

Oracle Collapse of κ: Anti-Relativization Stress Test for the Coarse-KW Invariant (A4)

Entry ID: `a81b4af0bc2b` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-04-27 17:34:08 UTC

1. Conjecture

Field A (mathematical branch): Coarse Geometric Karchmer-Wigderson (CG-KW) framework; underlying math: coarse geometry / Roe C-*algebras and metric geometry of boolean functions, with Mendel–Naor style metric-invariant obstructions* ***Field B*** *(complexity object): Relativized boolean formula depth / NC^1 vs P^A separation barriers — specifically, the complexity-theoretic object 'oracle-augmented formula depth d^A(f) over the basis B ∪ {f-gate}', and the induced uniform Roe algebra C_u^[X_{f^A}] viewed as a complexity invariant*

Statement:

> Let f:{0,1}^n → {0,1} be any boolean function, and let f^A denote the oracle-augmented variant whose KW-metric d_{f^A} is built from depth-bounded distinguishers that may use a single oracle gate computing f. Then (i) d_{f^A}(x,y) ≤ 1 for every (x,y) ∈ f^{-1}(0) × f^{-1}(1); (ii) the resulting controlled cover U has propagation R ≤ 1, so the algebraic uniform Roe algebra C_u^*[X_{f^A}] is the full matrix algebra on X_{f^A}; (iii) $HX^{1(X\_\{f}A\})$ = 0 because every coarse 1-cochain is a coboundary on a diameter-1 space; hence (iv) the trace pairing ⟨c,T⟩ vanishes for all admissible (c,T), forcing κ(f^A) = O(1) uniformly in n, while κ(f) can be ω(1). This realizes axiom A4: the coarse depth κ is destroyed by oracle access and therefore cannot relativize.

Rationale (proposer's reasoning):

> This isolates and tests axiom A4 (anti-relativization). The conjecture should hold because the KW-metric is *defined* via depth-bounded distinguishers; an oracle gate for f trivially separates f^{-1}(0) from f^{-1}(1) in one step, collapsing all nontrivial distances to 1. Once the metric collapses to a 1-bounded space, the Roe cohomology HX^1 becomes a finite-dimensional coboundary space and the index pairing tr(c·T) is forced to zero, so κ degenerates. Confirming this empirically validates that κ is a genuine metric invariant (not algebraic), distinguishing it from relativizing measures like communication complexity, and aligning with Mendel–Naor metric-cotype obstructions (A4) and Yu-style coarse rigidity (A5, used here only as the trace formalism).

Taxonomy category: `KARCHMER_WIGDERSON` (status at proposal time:)

Framework membership: framework `fw_85a254b4a0`, role: elaboration

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `601f7ebc71b9eba2`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> Across all f ∈ {AND_3, MAJ_3, PARITY_4, PARITY_5} (deterministic, 1 seed each since protocol is exact/non-stochastic), require: (a) d_{f^A}(x,y)=1 for every (x,y)∈f^{-1}(0)×f^{-1}(1); (b) dim HX^1(X_{f^A})=0; (c) κ̂(f^A)≤2; (d) κ̂(f)>κ̂(f^A)+0.5 on ≥1 function.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Karchmer-Wigderson coarse geometry Roe algebra formula depth` - `relativization barrier NC1 oracle formula depth metric invariant` - `uniform Roe algebra boolean function complexity propagation coarse cohomology`

Top relevant hits considered: - [http://arxiv.org/abs/2107.05128v2] Karchmer-Wigderson Games for Hazard-free Computation - [http://arxiv.org/abs/2312.08907v2] Roe algebras of coarse spaces via coarse geometric modules - [http://arxiv.org/abs/2002.07444v1] Multiparty Karchmer-Wigderson Games and Threshold Circuits - [http://arxiv.org/abs/2112.03677v7] On Baker-Gill-Solovay Oracle Turing Machines and Relativization Barrier - [http://arxiv.org/abs/2601.22054v1] MetricAnything: Scaling Metric Depth Pretraining with Noisy Heterogeneous Sources - [http://arxiv.org/abs/2412.00143v1] Is Oracle Pruning the True Oracle? - [http://arxiv.org/abs/2109.13292v1] The Hochschild Cohomology of Uniform Roe Algebras - [http://arxiv.org/abs/2501.00685v1] A quantization of coarse spaces and uniform Roe algebras

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
import sys
import json

def run_trial(seed: int) -> dict:
    random.seed(seed)
```

```
def AND_3(x):
    return x[0] and x[1] and x[2]

def MAJ_3(x):
    return sum(x) >= 2

def PARITY_4(x):
    return sum(x) % 2 == 1

def PARITY_5(x):
    return sum(x) % 2 == 1

functions = [AND_3, MAJ_3, PARITY_4, PARITY_5]

results = []
for f in functions:
    n = random.choice([5, 8, 11, 14])
    X_f = [(i, j) for i in range(2**n) for j in range(2**n) if f(i) != f(j)]

    def d_f(x, y):
        for k in range(1, n+1):
            queries = []
            for i in range(n):
                queries.append((i, x[i] ^ y[i]))
            if all(f(query[0]) == query[1] for query in queries):
                return math.log2(k)
        return float('inf')

    def d_fA(x, y):
        if f(x) != f(y):
            return 1
        else:
            return float('inf')

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i+1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            factor = A[i][i]
            for j in range(n):
                A[i][j] /= factor
            for j in range(m):
                if j != i:
                    factor = A[j][i]
                    for k in range(n):
                        A[j][k] -= factor * A[i][k]
        return A

    def compute_roe_cohomology(d_f, n):
        G = [[d_f(i, j) for j in range(2**n)] for i in range(2**n)]
        delta = []
        for i in range(1, 2**n):
```

```
            row = [0] * (2**n)
            row[i-1] = -1
            for j in range(i+1, 2**n):
                if d_f(i, j) == 1:
                    row[j-1] = 1
            delta.append(row)
        delta = gaussian_elimination(delta)
        rank = sum(1 for row in delta if any(x != 0 for x in row))
        return rank

    def compute_kappa_hat(f, d_f, n):
        max_value = float('-inf')
        for R in range(1, n+1):
            T = [[0] * (2**n) for _ in range(2**n)]
            for i in range(2**n):
                for j in range(2**n):
                    if abs(d_f(i, j)) <= R:
                        T[i][j] = 1
            c = []
            for e in range(1, 2**n):
                row = [0] * (2**n)
                row[e-1] = -1
                for f in range(e+1, 2**n):
                    if d_f(e, f) == 1:
                        row[f-1] = 1
                c.append(row)
            c = gaussian_elimination(c)
            rank_c = sum(1 for row in c if any(x != 0 for x in row))
            value = math.log(abs(sum(T[i][j] * c[j][i] for i in range(2**n) for j in range(2**
            max_value = max(max_value, value)
        return max_value

    d_f_values = [(x, y, d_f(x, y)) for x, y in X_f]
    d_fA_values = [(x, y, d_fA(x, y)) for x, y in X_f]

    HX1_fA = compute_roe_cohomology(d_fA, n)
    kappa_hat_f = compute_kappa_hat(f, d_f, n)
    kappa_hat_fA = compute_kappa_hat(lambda x: f(x), d_fA, n)

    results.append({
        "f": f.__name__,
        "n": n,
        "d_f_values": d_f_values,
        "d_fA_values": d_fA_values,
        "HX1_fA": HX1_fA,
        "kappa_hat_f": kappa_hat_f,
        "kappa_hat_fA": kappa_hat_fA
    })

total_kappa_hat_f = sum(result["kappa_hat_f"] for result in results)
total_kappa_hat_fA = sum(result["kappa_hat_fA"] for result in results)
avg_kappa_hat_f = total_kappa_hat_f / len(results)
avg_kappa_hat_fA = total_kappa_hat_fA / len(results)

conjecture_holds = all(result["HX1_fA"] == 0 and result["kappa_hat_fA"] <= 2 for result in res
```

```
    return {
        "metric_name": "κ̂",
        "metric_value": avg_kappa_hat_f,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else "mapping_undefined"
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [11, 23, 37, 53, 71]

    for seed in seeds:
        result = run_trial(seed)
        print(json.dumps({"SEED": seed, **result}))

    avg_kappa_hat_f = sum(result["kappa_hat_f"] for result in results) / len(results)
    avg_kappa_hat_fA = sum(result["kappa_hat_fA"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["HX1_fA"] == 0 and result["kappa_hat_

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={avg_kappa_hat_f} std=0 support_fraction={support_fraction}
    elif any(result["HX1_fA"] != 0 or result["kappa_hat_fA"] > 2 or result["kappa_hat_f"] <= resul
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={seeds[0
    else:
        print(f"RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0e4b213a.py", line 145, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0e4b213a.py", line 38, in run_trial
    X_f = [(i, j) for i in range(2**n) for j in range(2**n) if f(i) != f(j)]
                                                              ^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0e4b213a.py", line 22, in AND_3
    return x[0] and x[1] and x[2]
           ~^^^
TypeError: 'int' object is not subscriptable
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed with a TypeError (treating an int as subscriptable in AND_3) before producing any data, so none of the pre-registered support conditions could be evaluated. | next: Fix the boolean function input handling (decode the integer i into a bit-tuple via something like tuple((i»k)&1 for k in range(n)) before passing to f) and rerun the protocol.

11. Audit log (LLM calls)

Total LLM calls: 6

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	preregistration	claude_max	opus	0	0	8331
2	novelty	claude_max	opus	0	0	3134
3	novelty	claude_max	opus	0	0	9525
4	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15389
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18510
6	judge	claude_max	opus	0	0	4347

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 59236 ms total latency. Provider mix: {'claude_max': 4, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/a81b4af0bc2b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a81b4af0bc2b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/a81b4af0bc2b.tar.gz` (if generated)